

Atividade Pratica - Analise de Algoritmos de Ordenacao

Central de Distribuicao de Pedidos

Etapas 1 e 2: Criterio de Contagem

O codigo foi desenvolvido em arquivo separado (codigo_final.py). Nele, um unico vetor original e gerado para cada tamanho, e quatro copias identicas (.copy()) sao feitas a partir dele - uma para cada algoritmo - garantindo que todos recebam exatamente os mesmos dados.

Criterio de contagem utilizado em cada algoritmo:

- Bubble Sort: uma comparacao a cada verificacao $v[j] > v[j+1]$; uma troca a cada inversao de posicao entre dois elementos vizinhos.
- Insertion Sort: uma comparacao a cada verificacao $v[j] > \text{chave}$ dentro do laco de insercao; uma movimentacao a cada deslocamento de elemento para abrir espaco, mais uma movimentacao final referente a colocacao da chave na posicao correta ($v[j+1] = \text{chave}$), mesmo quando nao houve deslocamento algum antes.
- Selection Sort: uma comparacao a cada verificacao $v[j] < v[\text{menor}]$ buscando o menor elemento do restante do vetor; uma troca apenas quando o menor encontrado e diferente da posicao atual ($i \neq \text{menor}$).
- Quick Sort: usa o particionamento de Hoare, com o pivo escolhido no elemento central do trecho (nao no ultimo elemento). Sao contadas duas comparacoes por par de ponteiros avaliado (uma ao mover o ponteiro i, outra ao mover o ponteiro j); uma movimentacao e contada somente quando os elementos de fato trocam de posicao ($i \neq j$).

Por usar o elemento central como pivo (em vez do ultimo elemento), este Quick Sort evita o pior caso classico que ocorre em vetores ja ordenados ou em ordem inversa, mantendo bom desempenho em qualquer organizacao inicial dos dados - o que fica evidente nos resultados do desafio adicional.

Etapas 3: Resultados (vetores aleatorios)

Tam.	Bubble-Comp	Bubble-Troc	Insert-Comp	Insert-Mov	Select-Comp	Select-Troc	Quick-Comp	Quick-Mov
10	45	23	30	32	45	7	45	9
20	190	86	104	105	190	16	101	22
1.000	499.500	251.970	252.964	252.969	499.500	996	12.339	2.374

Etapas 4: Analise dos Resultados

- Menor numero de comparacoes com 10 elementos? Insertion Sort (30), pois para de comparar assim que encontra a posicao correta da chave, enquanto os demais sempre percorrem todo o trecho restante.
- Menos trocas/movimentacoes? Selection Sort (7), ja que so troca uma vez por posicao, apos confirmar o menor elemento do restante do vetor.
- O comportamento com 20 elementos foi parecido com o de 10? Sim, a ordem entre os algoritmos se manteve (Insertion com menos comparacoes, Selection com menos trocas), e as diferencas entre eles ja comecaram a crescer.
- O que aconteceu com 1.000 elementos? Bubble e Selection dispararam para quase 500 mil comparacoes, Insertion foi a quase 253 mil, enquanto o Quick Sort ficou em apenas 12.339 - uma diferenca de mais de 40 vezes em relacao aos demais.
- Bubble, Insertion e Selection tem complexidade $O(n^2)$, mas apresentaram exatamente o mesmo numero de operacoes? Nao. Com 1.000 elementos, Bubble e Selection empataram em comparacoes (499.500), mas Insertion fez bem menos (252.964), pois seu laco interno para assim que encontra a posicao correta. A notacao $O(n^2)$ descreve apenas a taxa de crescimento, nao o numero exato de operacoes de cada algoritmo.
- Maior crescimento no numero de operacoes? Bubble Sort e Selection Sort, que sempre fazem $n(n-1)/2$ comparacoes independentemente da ordem dos dados, atingindo o limite teorico de $O(n^2)$ mesmo em vetores aleatorios.
- Como o Quick Sort se diferenciou? Seu numero de comparacoes cresceu de forma muito mais lenta que o quadrado do tamanho da entrada, proximo de $O(n \log n)$, o que gerou uma vantagem cada vez maior sobre os demais conforme o vetor cresceu.
- Os resultados sao coerentes com a teoria? Sim. Os tres algoritmos $O(n^2)$ cresceram de forma quadratica, e o Quick Sort, com complexidade media $O(n \log n)$, manteve um numero de operacoes bem menor mesmo com 1.000 elementos.
- Qual algoritmo escolher para ordenar milhares de pedidos? O Quick Sort, pois o experimento mostrou que ele realiza muito menos operacoes que os demais conforme o volume de dados cresce, resultando em menor tempo de

processamento em um sistema que lida com grandes quantidades de pedidos diariamente.

Desafio Adicional: Aleatorio x Ordenado x Ordem Inversa (1.000 elementos)

Caso	Bubble-Comp	Bubble-Troc	Insert-Comp	Insert-Mov	Select-Comp	Select-Troc	Quick-Comp	Quick-Mov
Aleatorio	499.500	240.907	241.902	241.906	499.500	993	13.281	2.281
Ordenado	499.500	0	999	999	499.500	0	9.124	44
Ordem inversa	499.500	499.452	499.498	500.451	499.500	523	9.125	544

A organizacao inicial dos dados interfere na quantidade de operacoes da mesma forma para todos os algoritmos?

Nao, cada algoritmo reage de um jeito diferente:

- Bubble Sort: o numero de comparacoes se mantem em 499.500 em qualquer organizacao (essa implementacao nao tem parada antecipada), mas o numero de trocas varia bastante: zero no vetor ordenado, e quase 500 mil no vetor invertido (pior caso).
- Insertion Sort: e o que mais se beneficia de dados ja ordenados, caindo para apenas 999 comparacoes e 999 movimentacoes (proximo de $O(n)$); no vetor invertido, atinge seu pior caso, com quase 500 mil comparacoes.
- Selection Sort: mantem sempre 499.500 comparacoes, independente da organizacao inicial, pois sempre percorre o restante do vetor procurando o menor valor; apenas o numero de trocas muda.
- Quick Sort: como usa o elemento central como pivo, mantem um desempenho bom e estavel nos tres casos (13.281, 9.124 e 9.125 comparacoes), sem sofrer o pior caso quadratico que ocorreria se o pivo fosse sempre o primeiro ou o ultimo elemento do trecho.

Conclusao: a organizacao inicial dos dados pode melhorar bastante o desempenho de alguns algoritmos (Insertion Sort em vetor ja ordenado) e nao afetar outros da mesma forma (Selection Sort, que sempre faz o mesmo numero de comparacoes). A escolha do pivo no Quick Sort tambem influencia diretamente sua sensibilidade a organizacao dos dados - a versao usada aqui (pivo central) se mostrou mais robusta que a versao com pivo fixo no ultimo elemento.